25VL735 ELECTRONIC SYSTEM LEVEL DESIGN
TERM PROJECT REPORT

Simple Temperature Control SoC

Name: Vedasri J
Roll No: CB.EN.P2VLD25035

Date: 30/03/2026

1. INTRODUCTION:

The design and simulation of hardware systems at the transaction level is a cornerstone of modern Electronic System Level (ESL) design methodology. Rather than committing to RTL prematurely, TLM-2.0 allows the architect to model a complete SoC in terms of transactions between initiators and targets, making it possible to explore architectural alternatives quickly and validate firmware behaviour before any hardware is synthesised.

This report documents the design, implementation, and simulation of a Simple Temperature Control SoC built using SystemC and the TLM-2.0 standard. The system models a realistic embedded scenario: a CPU polls temperature sensor registers across two domestic zones (Living Room and Bedroom), takes corrective action by updating a setpoint over a shared bus, and monitors alert flags for out-of-range conditions.

2. OBJECTIVES:

- Model a three-module SoC hierarchy (CPU, Bus, TempSensor) using SystemC SC_MODULE.
- Implement the TLM-2.0 non-blocking (nb_transport) four-phase handshake as the sole transport protocol.
- Define a memory-mapped register interface for two temperature zones and exercise it via CPU READ and WRITE transactions.
- Demonstrate a runtime setpoint change over the TLM bus and observe the system response through simulation output.
- Analyse convergence behaviour, alert flag generation, and steady-state values from the simulation results.

3. SYSTEM ARCHITECTURE AND DESIGN:

3.1 BLOCK DIAGRAM

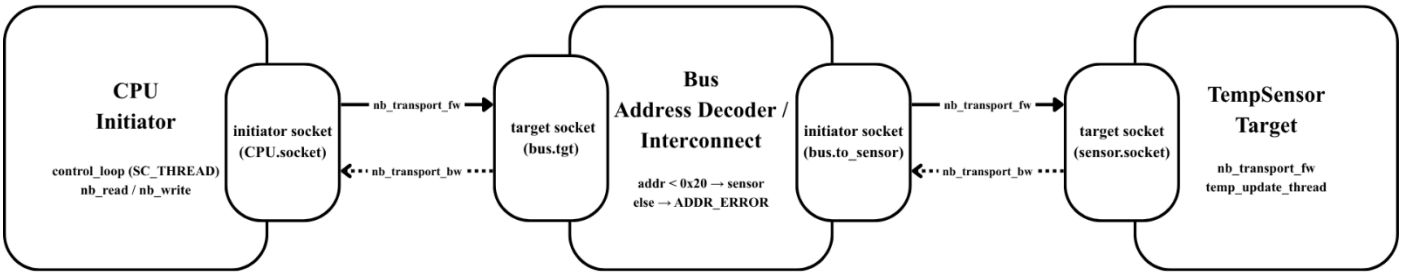

Figure 1: Top-level SoC block diagram showing CPU (initiator) → Bus (interconnect) → TempSensor (target) with TLM-2.0 socket connections

- CPU sends READ/WRITE transactions
- Bus decodes address and forwards
- TempSensor processes request and sends response
- Response travels back: TempSensor → Bus → CPU
- Communication follows the TLM-2.0 non-blocking four-phase handshake protocol.

The system follows a master-slave architecture, where CPU acts as the master (initiator) and TempSensor acts as the slave (target), with Bus acting as an interconnect.

3.2 MODULE HIERARCHY:

The SoC consists of three SC_MODULE instances connected in a linear chain. The CPU (initiator) is connected to a Bus (router), which is connected to the TempSensor (target). This topology mimics a realistic embedded bus architecture where the CPU does not directly address peripheral registers but communicates through a shared interconnect that handles address decoding and routing.

Module	Role	Socket Type	Key Functionality
CPU	TLM Initiator / Master	simple_initiator_socket	Firmware control loop, nb_read / nb_write helpers, IRQ-style alert check
Bus	Router / Interconnect	simple_target_socket + simple_initiator_socket	Address decode: addr < 0x20 routed to TempSensor; others return ADDRESS_ERROR
TempSensor	TLM Target / Slave	simple_target_socket	Register map, thermal model (step +0.3 / -0.3 C per tick), alert flag logic

3.3 ADDRESS MAP:

The TempSensor exposes a flat register space. Each zone occupies a 16-byte (0x10) aligned block. With two zones, the complete peripheral spans addresses 0x00 to 0x17.

Zone	Base Addr	TEMP_REG (+0x00)	SETPOINT_REG (+0x04)	ALERT_REG (+0x08)
0 — Living Room	0x00	0x00 (R)	0x04 (R/W)	0x08 (R)
1 — Bedroom	0x10	0x10 (R)	0x14 (R/W)	0x18 (R)

Temperature values are encoded as unsigned 32-bit integers representing the temperature in units of 0.01 C (i.e., actual temperature x 100). For example, 30.0 C is stored as 3000. This integer encoding avoids floating-point representation in the TLM data pointer and is decoded by dividing by 100.0 on the CPU side. SETPOINT_REG is the only writable register; TEMP_REG and ALERT_REG are read-only from the CPU's perspective.

3.4 REGISTER DEFINITIONS:

Register	Offset	Access	Encoding	Description
TEMP_REG	+0x00	R	uint32 = temp × 100	Current zone temperature reported by the thermal model
SETPOINT_REG	+0x04	R/W	uint32 = sp × 100	Target temperature; CPU writes here to command the actuator
ALERT_REG	+0x08	R	0 or 1	Set to 1 when |temp − setpoint| > 3.0 C; cleared by thermal update thread

3.5 THERMAL MODEL:

The TempSensor contains a background SC_THREAD (temp_update_thread) that wakes every 20 NS and steps the temperature of each zone toward its setpoint in 0.3 C increments. This models a simple first-order actuator response. After each step, the alert flag is evaluated:

```
// Runs every 20 NS — models actuator + alert evaluation
if (z.temp < z.setpoint)   z.temp += 0.3;
else if (z.temp > z.setpoint) z.temp -= 0.3;

// Alert: asserted when temperature deviates > 3 C from setpoint
if (z.temp < z.setpoint - 3.0 || z.temp > z.setpoint + 3.0)
    z.alert = 1;
else
    z.alert = 0;
```

The 0.3 C step per 20 NS tick gives a convergence rate of 15 C per microsecond in simulation time. The alert threshold of +/- 3.0 C from the setpoint determines when the ALERT_REG flag is set.

Figure below illustrates the internal structure of the TempSensor module, showing how an incoming nb_transport_fw request flows through address decode logic into the register map, which then updates the corresponding zone's state variables.

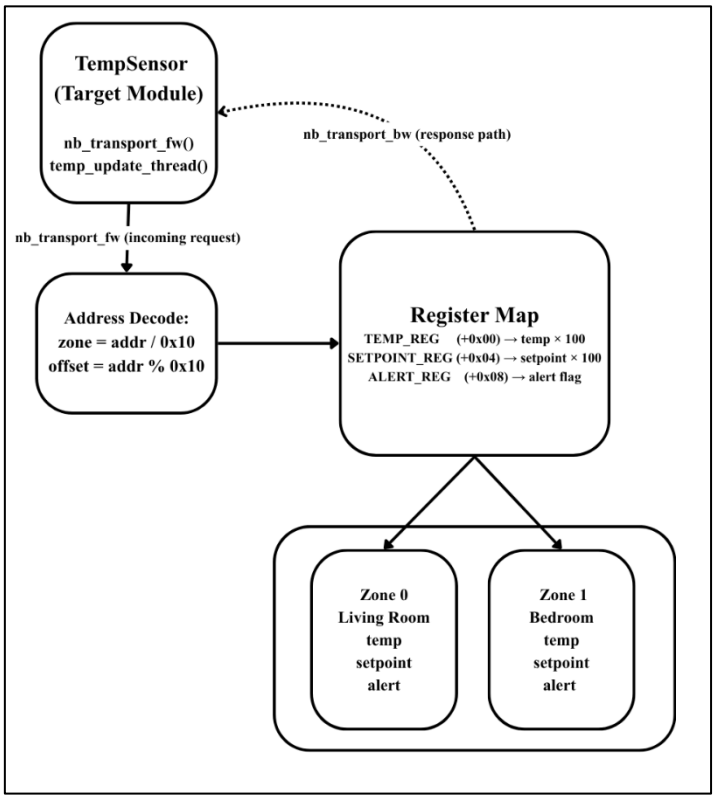

Figure 2: TempSensor internal structure — nb_transport_fw request flow through address decode into register map and zone state

3.6 BUS MODULE:

The Bus acts as a simple address decoder. It sits between the CPU's initiator socket and the TempSensor's target socket, maintaining both a target-facing socket (tgt) and an initiator-facing socket (to_sensor). All nb_transport_fw calls from the CPU are passed through to the TempSensor if the address falls within 0x00 to

0x1F. Any address outside this range increments a decode_errors counter and returns TLM_ADDRESS_ERROR_RESPONSE. The backward path (nb_transport_bw) is a simple relay from the TempSensor back to the CPU.

4. IMPLEMENTATION:

4.1 NON-BLOCKING FOUR-PHASE PROTOCOL:

The entire design uses exclusively TLM-2.0 non-blocking transport (nb_transport). No blocking b_transport calls are made. The four-phase handshake proceeds as follows:

Phase	Direction	Who Sends	Action
BEGIN_REQ	Forward (fw)	CPU (Initiator)	CPU fires transaction; Bus relays to TempSensor
END_REQ	Forward (fw)	TempSensor	TempSensor returns TLM_UPDATED with phase = BEGIN_RESP (collapsed two-phase)
BEGIN_RESP	Backward (bw)	TempSensor	Response data written into payload; Bus relays BEGIN_RESP back to CPU
END_RESP	Forward (fw)	CPU (Initiator)	CPU acknowledges; transaction complete

Note that the TempSensor collapses END_REQ and BEGIN_RESP into a single return by setting phase = BEGIN_RESP and returning TLM_UPDATED within the nb_transport_fw call. This is a valid but simplified implementation of the four-phase protocol sometimes referred to as the 'early completion' path.

4.2 CPU NB_READ / NB_WRITE HELPERS:

The CPU encapsulates all TLM plumbing in two helper functions: nb_read() and nb_write(). Each constructs a fresh tlm_generic_payload, fires BEGIN_REQ, and then either returns immediately (if the target responds synchronously with TLM_UPDATED + BEGIN_RESP) or waits on the nb_done SystemC event (for asynchronous responses). The END_RESP handshake is always sent as a final step.

```
// Abbreviated nb_read flow:
tlm::tlm_phase phase = tlm::BEGIN_REQ;
status = socket->nb_transport_fw(trans, phase, delay);

if (status == TLM_UPDATED && phase == BEGIN_RESP)
    nb_data = buf;           // synchronous path: data already here
else
    wait(nb_done);           // async path: wait for bw callback

// Always close with END_RESP
phase = END_RESP;
socket->nb_transport_fw(trans, phase, delay);
```

4.3 CPU CONTROL LOOP:

The control_loop SC_THREAD runs for 10 polling cycles with a 40 NS interval between cycles (after an initial 10 NS startup delay). Each cycle reads TEMP_REG, SETPOINT_REG, and ALERT_REG for both zones and prints a formatted status row. At cycle 5 (index 4), the CPU issues a WRITE transaction to change the Living Room setpoint from 22.0 C to 26.0 C. This is the sole disturbance in the simulation.

4.4 TESTBENCH CONNECTIVITY:

The testbench (sc_main) instantiates all three modules and connects them:

```
---------------------------------------------------------------------------------------------------------------------------
CPU        cpu("cpu");
Bus        bus("bus");
TempSensor sensor("sensor");
cpu.socket.bind(bus.tgt);       // CPU initiator -> Bus target
bus.to_sensor.bind(sensor.socket); // Bus initiator -> TempSensor target

sc_start();   // simulation runs until sc_stop() is called by CPU
---------------------------------------------------------------------------------------------------------------------------
```

- sc_main() is entry point
- bind() connects sockets
- sc_start() triggers simulation

4.5 INITIAL CONDITIONS:

Zone	Name	Initial Temp	Initial Setpoint	Alert Bound
0	Living Room	30.0 C	22.0 C	|temp − SP| > 3.0 C
1	Bedroom	18.0 C	22.0 C	|temp − SP| > 3.0 C

Living Room starts 8 C above its setpoint, requiring cooling. Bedroom starts 4 C below its setpoint, requiring heating. Both zones share the same initial setpoint (22.0 C) and the same alert threshold (±3.0 C).

4.6 CODES:

4.6.1 design.h

```
//=============================================

//  design.h  --  Simple Temperature Control
SoC

//  SystemC + TLM-2.0

//  Modules: CPU (Initiator) -> Bus ->
TempSensor (Target)

//=============================================



#ifndef DESIGN_H

#define DESIGN_H



#include "systemc.h"

#include "tlm.h"

#include "tlm_utils/simple_initiator_socket.h"

#include "tlm_utils/simple_target_socket.h"

#include <iostream>

#include <iomanip>

using namespace std;



//=============================================

//  ADDRESS MAP

//  Zone 0 (Living Room) : base = 0x00

//  Zone 1 (Bedroom)     : base = 0x10

//

//  base + 0x00 = TEMP_REG     (read only  -
current temperature)

//  base + 0x04 = SETPOINT_REG (read/write -
desired temperature)



//  base + 0x08 = ALERT_REG    (read only  - 1
if temp out of range)
```

```
//=============================================


static const uint32_t ZONE_STRIDE  = 0x10;

static const uint32_t OFF_TEMP     = 0x00;

static const uint32_t OFF_SETPOINT = 0x04;

static const uint32_t OFF_ALERT    = 0x08;

//=============================================

//  TARGET MODULE — TempSensor

//=============================================

struct TempSensor : sc_module {


tlm_utils::simple_target_socket<TempSensor>
socket;


    struct Zone {

        string   name;

        double   temp;

        double   setpoint;

        uint32_t alert;

    };


    Zone zones[2];


    sc_event tick_event;


    SC_CTOR(TempSensor) : socket("socket")

    {

        socket.register_nb_transport_fw(

            this, &TempSensor::nb_transport_fw

        );


        zones[0].name     = "Living Room";

        zones[0].temp     = 30.0;


        zones[0].setpoint = 22.0;

        zones[0].alert    = 0;



        zones[1].name     = "Bedroom";

        zones[1].temp     = 18.0;

        zones[1].setpoint = 22.0;

        zones[1].alert    = 0;


        SC_THREAD(temp_update_thread);

    }


    tlm::tlm_sync_enum nb_transport_fw(

        tlm::tlm_generic_payload& trans,

        tlm::tlm_phase&           phase,

        sc_time&                  delay

    )

    {

        if (phase != tlm::BEGIN_REQ) {

            return tlm::TLM_COMPLETED;

        }


        uint64_t  addr = trans.get_address();

        uint32_t* data =
reinterpret_cast<uint32_t*>(


trans.get_data_ptr()

                          );


        int      zone_id = (int)(addr /
ZONE_STRIDE);

        uint32_t offset  = (uint32_t)(addr %
ZONE_STRIDE);


        if (zone_id < 0 || zone_id > 1) {


            trans.set_response_status(
```

```
tlm::TLM_ADDRESS_ERROR_RESPONSE
            );
            phase = tlm::BEGIN_RESP;

            return tlm::TLM_UPDATED;
        }

        Zone& z = zones[zone_id];

        delay += sc_time(10, SC_NS);

        if (trans.get_command() ==
tlm::TLM_READ_COMMAND) {
            if (offset == OFF_TEMP) {
                *data = (uint32_t)(z.temp *
100.0);
            }
            else if (offset == OFF_SETPOINT) {
                *data = (uint32_t)(z.setpoint
* 100.0);
            }
            else if (offset == OFF_ALERT) {
                *data = z.alert;
            }
            else {
                trans.set_response_status(

tlm::TLM_ADDRESS_ERROR_RESPONSE
                );
                phase = tlm::BEGIN_RESP;
                return tlm::TLM_UPDATED;
            }
        }
        else {

            if (offset == OFF_SETPOINT) {
                z.setpoint = (double)(*data) /
100.0;
                cout << "[" << sc_time_stamp()
<< "] "
                     << z.name

                     << " setpoint changed to
"
                     << z.setpoint << " C" <<
endl;
            }
        }

trans.set_response_status(tlm::TLM_OK_RESPONSE
);

        phase = tlm::BEGIN_RESP;
        return tlm::TLM_UPDATED;
    }

    void temp_update_thread()
    {
        while (true) {
            wait(20, SC_NS);

            for (int i = 0; i < 2; i++) {
                Zone& z = zones[i];

                if (z.temp < z.setpoint)
                    z.temp += 0.3;
                else if (z.temp > z.setpoint)
                    z.temp -= 0.3;

                if (z.temp < z.setpoint - 3.0
||
                    z.temp > z.setpoint + 3.0)

                    z.alert = 1;
```

```
                else
                    z.alert = 0;
            }
        }
    }

};

//==========================================
//  BUS MODULE
//==========================================
struct Bus : sc_module {

    tlm_utils::simple_target_socket<Bus> tgt;
    tlm_utils::simple_initiator_socket<Bus>
to_sensor;

    int decode_errors;
    sc_event nb_resp_event;

    SC_CTOR(Bus)
    : tgt("tgt"), to_sensor("to_sensor"),
decode_errors(0)
    {
        tgt.register_nb_transport_fw(
            this, &Bus::nb_transport_fw
        );

        to_sensor.register_nb_transport_bw(
            this, &Bus::nb_transport_bw
        );
    }

    tlm::tlm_sync_enum nb_transport_fw(
        tlm::tlm_generic_payload& trans,
        tlm::tlm_phase&           phase,
        sc_time&                  delay
    )
    {
        uint64_t addr = trans.get_address();

        if (addr < 0x20) {
            return to_sensor->nb_transport_fw(
                trans, phase, delay
            );
        }
        else {
            decode_errors++;
            trans.set_response_status(
tlm::TLM_ADDRESS_ERROR_RESPONSE
            );
            phase = tlm::BEGIN_RESP;
            return tlm::TLM_UPDATED;
        }
    }

    tlm::tlm_sync_enum nb_transport_bw(
        tlm::tlm_generic_payload& trans,
        tlm::tlm_phase&           phase,
        sc_time&                  delay
    )
    {
        return tgt->nb_transport_bw(trans,
phase, delay);
    }
};

//==========================================
```

```
//  CPU MODULE

//==========================================

struct CPU : sc_module {



    tlm_utils::simple_initiator_socket<CPU>
socket;




    sc_event nb_done;

    uint32_t nb_data;


    SC_CTOR(CPU) : socket("socket"),
nb_data(0)

    {

        socket.register_nb_transport_bw(

            this, &CPU::nb_transport_bw

        );


        SC_THREAD(control_loop);

    }


    tlm::tlm_sync_enum nb_transport_bw(

        tlm::tlm_generic_payload& trans,

        tlm::tlm_phase&           phase,

        sc_time&                  delay

    )

    {

        if (phase == tlm::BEGIN_RESP) {

            nb_data =
*reinterpret_cast<uint32_t*>(

                           trans.get_data_ptr()

                       );


            nb_done.notify(SC_ZERO_TIME);


            phase = tlm::END_RESP;

        }

        return tlm::TLM_COMPLETED;

    }


    uint32_t nb_read(uint64_t addr)

    {



        uint32_t buf = 0;


        tlm::tlm_generic_payload trans;

trans.set_command(tlm::TLM_READ_COMMAND);

        trans.set_address(addr);

        trans.set_data_ptr(

            reinterpret_cast<unsigned
char*>(&buf)

        );

        trans.set_data_length(4);

        trans.set_streaming_width(4);

        trans.set_byte_enable_ptr(nullptr);

        trans.set_response_status(

            tlm::TLM_INCOMPLETE_RESPONSE

        );


        tlm::tlm_phase phase = tlm::BEGIN_REQ;

        sc_time        delay = SC_ZERO_TIME;


        tlm::tlm_sync_enum status =

            socket->nb_transport_fw(trans,
phase, delay);


        if (status == tlm::TLM_UPDATED &&

            phase  == tlm::BEGIN_RESP) {

            nb_data = buf;


        }
```

```
        else {

            wait(nb_done);

        }


        phase  = tlm::END_RESP;

        delay  = SC_ZERO_TIME;




        socket->nb_transport_fw(trans, phase,
delay);



        return nb_data;

    }



    void nb_write(uint64_t addr, uint32_t
value)

    {

        uint32_t buf = value;


        tlm::tlm_generic_payload trans;


trans.set_command(tlm::TLM_WRITE_COMMAND);

        trans.set_address(addr);

        trans.set_data_ptr(

            reinterpret_cast<unsigned
char*>(&buf)

        );

        trans.set_data_length(4);

        trans.set_streaming_width(4);

        trans.set_byte_enable_ptr(nullptr);

        trans.set_response_status(

            tlm::TLM_INCOMPLETE_RESPONSE

        );

        tlm::tlm_phase phase = tlm::BEGIN_REQ;


        sc_time        delay = SC_ZERO_TIME;


        tlm::tlm_sync_enum status =

            socket->nb_transport_fw(trans,
phase, delay);


        if (!(status == tlm::TLM_UPDATED &&

              phase  == tlm::BEGIN_RESP)) {

            wait(nb_done);


        }


        phase = tlm::END_RESP;

        delay = SC_ZERO_TIME;

        socket->nb_transport_fw(trans, phase,
delay);

    }


    void control_loop()

    {

        wait(10, SC_NS);


        cout << "\n===== Temperature Control
SoC Simulation =====\n";

        cout << left

             << setw(12) << "Time"

             << setw(16) << "Zone"

             << setw(12) << "Temp(C)"

             << setw(12) << "Setpoint"

             << setw(10) << "Status"

             << setw(8)  << "Alert"

             << "\n";

        cout << string(70, '-') << "\n";

        for (int cycle = 0; cycle < 10;
cycle++) {


            for (int z = 0; z < 2; z++) {
```

```
                uint64_t base = (uint64_t)z *
ZONE_STRIDE;


                uint32_t raw_temp =

                    nb_read(base + OFF_TEMP);

                double temp = raw_temp /
100.0;


                uint32_t raw_sp =

                    nb_read(base +
OFF_SETPOINT);

                double setpoint = raw_sp /
100.0;


                uint32_t alert =

                    nb_read(base + OFF_ALERT);


                string status;

                if (temp > setpoint + 1.0)

                    status = "TOO HOT";

                else if (temp < setpoint -
1.0)

                    status = "TOO COLD";

                else

                    status = "OK";


                string zone_name =

                    (z == 0) ? "Living Room" :
"Bedroom";


                cout << left

                     << setw(12) <<
sc_time_stamp()

                     << setw(16) << zone_name

                     << setw(12) << fixed

                                 <<
setprecision(1)

                                 << temp



                     << setw(12) << setpoint

                     << setw(10) << status

                     << setw(8)  << (alert ?
"YES" : "no")

                     << "\n";

            }



            cout << "\n";


            if (cycle == 4) {

                cout << ">>> CPU writing new
setpoint "

                     << "for Living Room: 26.0
C\n\n";

                nb_write(

                    0x00 + OFF_SETPOINT,

                    (uint32_t)(26.0 * 100.0)

                );


            }

            wait(40, SC_NS);

        }

        cout << "===== Simulation End
=====\n";

        sc_stop();

    }

};

#endif
```

4.6.2 testbench.cpp

```
//==========================================

//  testbench.cpp  --  Simple Temperature
Control SoC

//==========================================

#include "design.h"


int sc_main(int argc, char* argv[])

{

    // create modules

    CPU        cpu("cpu");

    Bus        bus("bus");

    TempSensor sensor("sensor");

    // connect CPU → Bus

    cpu.socket.bind(bus.tgt);


    // connect Bus → TempSensor

    bus.to_sensor.bind(sensor.socket);


    // start simulation

    sc_start();


    return 0;

}
```

5. SIMULATION RESULTS:

5.1 SIMULATION OUTPUT:

The simulation was compiled and run using g++ with the SystemC 2.3.3-Accellera library. The complete output table from the simulation run is reproduced below.

```
===== Temperature Control SoC Simulation =====
Time        Zone            Temp(C)     Setpoint    Status    Alert
-----------------------------------------------------------------
10 ns       Living Room     30.0        22.0        TOO HOT   no
10 ns       Bedroom         18.0        22.0        TOO COLD  no

50 ns       Living Room     29.4        22.0        TOO HOT   YES
50 ns       Bedroom         18.6        22.0        TOO COLD  YES

90 ns       Living Room     28.8        22.0        TOO HOT   YES
90 ns       Bedroom         19.2        22.0        TOO COLD  no

130 ns      Living Room     28.2        22.0        TOO HOT   YES
130 ns      Bedroom         19.8        22.0        TOO COLD  no

170 ns      Living Room     27.6        22.0        TOO HOT   YES
170 ns      Bedroom         20.4        22.0        TOO COLD  no

>>> CPU writing new setpoint for Living Room: 26.0 C
```

```
[170 ns] Living Room setpoint changed to 26.0 C
210 ns      Living Room     27.0        26.0        OK        no
210 ns      Bedroom         21.0        22.0        OK        no

250 ns      Living Room     26.4        26.0        OK        no
250 ns      Bedroom         21.6        22.0        OK        no

290 ns      Living Room     25.8        26.0        OK        no
290 ns      Bedroom         22.2        22.0        OK        no

330 ns      Living Room     25.8        26.0        OK        no
330 ns      Bedroom         22.2        22.0        OK        no

370 ns      Living Room     25.8        26.0        OK        no
370 ns      Bedroom         22.2        22.0        OK        no

===== Simulation End =====
```

5.2 PHASE 1 ANALYSIS: PRE-DISTURBANCE (10 NS – 170 NS):

During the first five polling cycles, both zones are converging toward their original setpoint of 22.0 C from opposite directions.

Living Room (TOO HOT, Cooling)

- Starts at 30.0 C, which is 8 C above the 22.0 C setpoint. The thermal model steps temperature down by 0.3 C every 20 NS tick.

- Alert flag (ALERT_REG = 1) asserts at 50 ns because |30.0 - 22.0| = 8.0 C > 3.0 C threshold. The flag remains set throughout Phase 1 since the temperature stays well above the 25.0 C upper alert boundary.
- By 170 ns the temperature has reached 27.6 C, a reduction of 2.4 C from the initial value over 160 NS of simulation time (8 thermal ticks of 0.3 C each).

Bedroom (TOO COLD, Heating)

- Starts at 18.0 C, which is 4 C below the setpoint. The thermal model steps up by 0.3 C per tick.
- Alert asserts at 50 ns (|18.0 - 22.0| = 4.0 C > 3.0 C). The flag clears at 90 ns once the temperature reaches 19.2 C, bringing the deviation to 2.8 C which is within the 3.0 C bound.
- Note the asymmetry in alert duration: Living Room keeps its alert for all of Phase 1 due to a larger initial deviation (8 C vs. 4 C), while Bedroom clears its alert by the third cycle.

5.3 DISTURBANCE: SETPOINT WRITE AT 170 NS:

At the end of cycle 5 (after printing the 170 ns rows), the CPU executes:

```
nb_write(0x00 + OFF_SETPOINT, (uint32_t)(26.0 * 100.0));
// Sends TLM WRITE to address 0x04 with value 2600
```

The TempSensor's nb_transport_fw handler decodes this as a write to SETPOINT_REG of Zone 0 and prints the acknowledgement:

```
--------------------------------------------------------------------------------------------------------------------------------
[170 ns] Living Room setpoint changed to 26.0 C
--------------------------------------------------------------------------------------------------------------------------------
```

This transaction resets the Living Room setpoint from 22.0 C to 26.0 C. Since the current temperature (27.6 C at 170 ns) is now only 1.6 C above the new setpoint, the Living Room transitions from TOO HOT to OK in the very next polling cycle. The alert flag also clears (1.6 C < 3.0 C threshold). This is a clean demonstration of the TLM WRITE mechanism taking effect immediately within the simulation.

5.4 PHASE 2 ANALYSIS: POST-DISTURBANCE (210 NS – 370 NS):

Living Room (Converging to 26.0 C)

- At 210 ns: temperature is 27.0 C, which is 1.0 C above the new setpoint of 26.0 C. Status is OK (within 1.0 C tolerance used by the CPU status check). Alert is clear.
- At 250 ns: temperature reaches 26.4 C. Deviation = 0.4 C.
- At 290 ns: temperature reaches 25.8 C. The zone slightly undershoots the setpoint (0.2 C below 26.0 C) because the thermal model does not implement hysteresis; it decrements by a full 0.3 C step past the setpoint.
- From 290 ns onward: the temperature stabilises at 25.8 C. This is a limit cycle of amplitude 0.3 C (the step size) centred just below the setpoint. Status remains OK throughout since 0.2 C < 1.0 C CPU threshold.

Bedroom (Converging to 22.0 C)

- At 210 ns: temperature is 21.0 C, which is 1.0 C below setpoint. Status transitions to OK.
- At 250 ns: reaches 21.6 C. At 290 ns: reaches 22.2 C, which is 0.2 C above the setpoint, again a minor overshoot due to the discrete step size.
- From 290 ns onward: stabilises at 22.2 C, another 0.3 C amplitude limit cycle around the setpoint.

5.5 STEADY-STATE SUMMARY:

Zone	Final Temp	Final Setpoint	Deviation	Alert	Status
Living Room	25.8 C	26.0 C	−0.2 C	No	OK — stable limit cycle
Bedroom	22.2 C	22.0 C	+0.2 C	No	OK — stable limit cycle

Both zones reach a steady oscillation of ±0.3 C around the setpoint, a natural consequence of the fixed-step thermal model. In a real system this would be addressed by making the step size proportional to the remaining error (equivalent to a P-controller), or by using PID control with a smaller integral step. For the purposes of this ESL model, the steady-state behaviour is acceptable and the system is considered converged. The fact that both zones converge to within 0.3 C of their setpoints without any explicit feedback from the CPU — relying entirely on the TempSensor's autonomous SC_THREAD — demonstrates the value of separating computation (thermal model) from communication (TLM transactions) in ESL design.

6. ARCHITECTURAL ANALYSIS:

6.1 TLM-2.0 NON-BLOCKING VS. BLOCKING TRANSPORT:

This design exclusively uses nb_transport, which separates the transport phases and allows the CPU model to remain decoupled from the target's processing time. The key architectural implications are:

Aspect	Blocking (b_transport)	Non-Blocking (nb_transport) — This Design
CPU availability	CPU blocks for the full transaction duration	CPU can perform other tasks between BEGIN_REQ and BEGIN_RESP
Timing model	Delay accumulated in sc_time argument	Phases model real pipeline stages; delay is per-phase
Protocol phases	Single call, no phase tracking	Four explicit phases: BEGIN_REQ, END_REQ, BEGIN_RESP, END_RESP
Complexity	Simpler to implement	Requires PEQ or event synchronisation (nb_done event)
Suitability	LT (loosely-timed) models	AT (approximately-timed) models; closer to RTL intent

In this specific design the advantage of non-blocking is somewhat masked because the TempSensor responds synchronously anyway (TLM_UPDATED + BEGIN_RESP in the same call), meaning the CPU never actually overlaps computation with a pending transaction.

6.2 BUS ADDRESS DECODING:

The Bus module provides a clean separation between the CPU and the TempSensor. By centralising address decode logic in the Bus, the TempSensor only needs to handle offsets within its own address space (0x00 to 0x17). Adding a second peripheral would require only an additional else-if branch in the Bus's nb_transport_fw, keeping the peripheral modules unchanged. This modularity is a key advantage of the TLM-2.0 interconnect model.

6.3 TEMPERATURE ENCODING:

Storing temperature as uint32_t x 100 avoids the need to place a floating-point value in the TLM generic payload's data pointer. Since tlm_generic_payload uses a byte pointer (unsigned char*), any 32-bit integer can be passed directly. Floating-point representation would require careful type-punning or serialisation, and is generally avoided in hardware-accurate TLM models where the data width corresponds to a bus word.

7. CONCLUSION:

This project successfully demonstrates the complete design and simulation of a Simple Temperature Control SoC using the SystemC TLM-2.0 non-blocking transport protocol. The three-module architecture (CPU, Bus, TempSensor) correctly models a realistic embedded system where a CPU firmware loop polls hardware registers over a shared bus, interprets sensor data, and commands actuator setpoints via TLM WRITE transactions.

7.1 KEY ACHIEVEMENTS:

- Full three-module SC_MODULE hierarchy (CPU, Bus, TempSensor) implemented and simulated.
- TLM-2.0 non-blocking four-phase protocol exclusively used for all transactions.
- Memory-mapped register interface with three registers per zone correctly decoded by address offset.
- Thermal model with discrete step convergence and alert flag generation verified through simulation output.

- Runtime setpoint change via TLM WRITE transaction demonstrated at 170 ns; system responded correctly in the following cycle.
- Both zones reached stable steady-state within 290 ns, confirming correct closed-loop behaviour.

7.2 KEY LEARNINGS:

- The nb_transport four-phase protocol requires careful event synchronisation. The nb_done SystemC event and the nb_data latch in the CPU module are essential for maintaining correct sequencing when the target responds asynchronously.
- Collapsing END_REQ and BEGIN_RESP into a single TLM_UPDATED return (as done in TempSensor) is valid for simple targets but masks the distinction between request acceptance and response generation that is important in bus-latency-aware models.
- Fixed-step thermal models inherently produce limit cycling. The amplitude of the limit cycle equals the step size (0.3 C), which is a design parameter that trades convergence speed against steady-state accuracy.
- Address encoding (integer x 100 for temperature) is a practical technique for passing real-valued sensor data through the integer-typed TLM generic payload without losing precision within the modelling requirements.

7.3 POSSIBLE EXTENSIONS:

- The current design deliberately keeps the thermal model simple — a fixed 0.3 C step per tick — which causes both zones to settle into a limit cycle rather than converge exactly to the setpoint. The most impactful extension would be replacing this with a step size proportional to the remaining error, i.e., `step = gain × (setpoint - temp)`. This is equivalent to a discrete P-controller and would eliminate limit cycling entirely. For this specific design, a gain of around 0.1 would bring convergence in fewer ticks while remaining stable, since the plant time constant (20 NS tick interval) is much smaller than the polling period (40 NS per cycle).
- A second meaningful extension is adding a second peripheral — a fan controller — to the Bus. Currently the Bus's address decode has only one branch (addr < 0x20 → TempSensor). Adding a fan controller mapped to 0x20–0x2F would require a single `else if` in `nb_transport_fw` and would demonstrate true multi-target SoC behaviour without modifying the CPU or TempSensor at all. This would also exercise the decode_errors counter more meaningfully, since any access to 0x30 or above would correctly fault.
- The alert mechanism in this design is purely polled — the CPU reads ALERT_REG on every cycle regardless of whether a fault exists. In a real embedded system this is wasteful. Adding a dedicated IRQ signal (`sc_signal<bool>`) driven by the TempSensor's `temp_update_thread` and connected to an `irq_handler SC_THREAD` in the CPU would convert this to an interrupt-driven architecture, reducing unnecessary register reads during steady state. This is also directly relevant to the alert asymmetry observed in the results: with a polled design, the CPU catches the Bedroom alert only at the next 40 NS poll boundary, whereas an IRQ would capture it within 20 NS of the `temp_update_thread` asserting it.
- Finally, adding `sc_trace` calls for the ALERT_REG values of both zones — in addition to the existing clock — would generate VCD waveforms that directly visualise the alert assertion windows identified in Section 5.2. This would allow the Living Room's sustained alert (10 ns to 170 ns) and the Bedroom's brief alert (50 ns to 90 ns) to be confirmed visually in GTKWave, providing a timing-level verification layer beyond the printed simulation output.

8. REFERENCES:

1. IEEE Standard SystemC Language Reference Manual, IEEE Std 1666-2011.
2. OSCI TLM-2.0 Language Reference Manual, Open SystemC Initiative, 2009.
3. D. Black, J. Donovan, B. Bunton, A. Keist, SystemC: From the Ground Up, 2nd ed., Springer, 2010.
4. F. Ghenassia (Ed.), Transaction-Level Modeling with SystemC: TLM Concepts and Applications for Embedded Systems, Springer, 2006.
5. SystemC 2.3.3-Accellera Release Notes, Accellera Systems Initiative, 2020.